

Lesson 3: Controllers and MVC Pattern

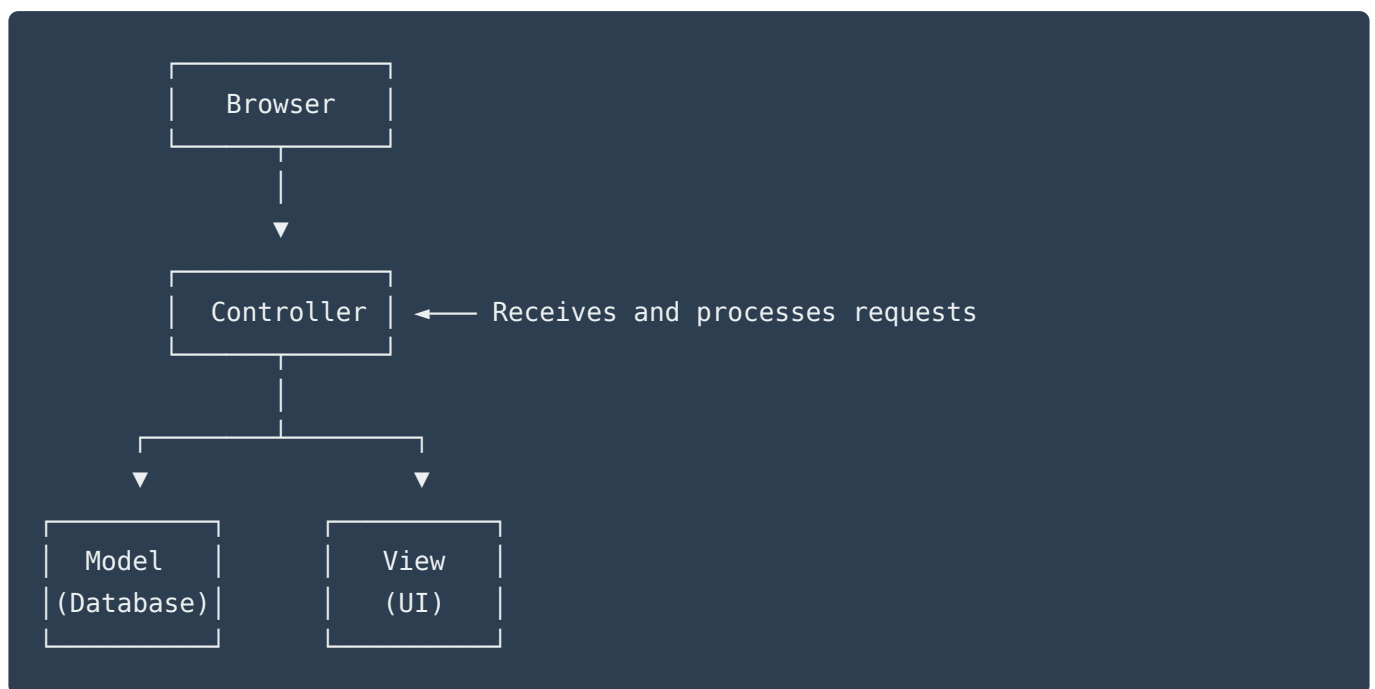
▢ Table of Contents

1. [Introduction to MVC Pattern](#)
 2. [What are Controllers?](#)
 3. [Creating Controllers](#)
 4. [Types of Controllers](#)
 5. [Resource Controllers](#)
 6. [Dependency Injection](#)
 7. [Middleware in Controllers](#)
 8. [Practical Exercises](#)
-

Introduction to MVC Pattern

What is MVC?

MVC stands for **Model-View-Controller**, an architectural pattern for organizing code.



MVC Components:

1. Model 📦

- Handles database operations
- Contains business logic
- Example: `User`, `Post`, `Product`

```
class Product extends Model
{
    public function getDiscountedPrice()
    {
        return $this->price * 0.9;
    }
}
```

2. View 📄

- User interface (HTML)
- Displays data to users
- Blade files in `resources/views`

```
<h1>{{ $product->name }}</h1>
<p>Price: {{ $product->price }}</p>
```

3. Controller 📦

- Links Model and View
- Receives requests
- Processes logic
- Returns responses

```
class ProductController extends Controller
{
    public function show($id)
    {
        $product = Product::find($id);
        return view('product.show', compact('product'));
    }
}
```

```
}
```

Why Use MVC?

□ Separation of Concerns

- Each part has a specific responsibility

□ Easy Maintenance

- Change UI without changing application logic

□ Reusability

- Use same Model in different Controllers

□ Team Collaboration

- One developer works on Views, another on Controllers
-

What are Controllers?

Definition

A Controller is a **PHP class** containing methods to handle HTTP requests.

Before Controllers - Direct Routes

```
// In routes/web.php
Route::get('/products', function () {
    $products = DB::table('products')->get();
    return view('products.index', compact('products'));
});

Route::get('/products/{id}', function ($id) {
    $product = DB::table('products')->find($id);
    return view('products.show', compact('product'));
});

Route::post('/products', function () {
    // 50 lines of code...
```

```
DB::table('products')->insert([...]);
return redirect('/products');
});
```

Problems:

- ☐ Duplicate code
- ☐ Hard to maintain
- ☐ Routes file becomes too large
- ☐ Difficult to reuse

After Controllers - Organized Solution

```
// In routes/web.php
Route::get('/products', [ProductController::class, 'index']);
Route::get('/products/{id}', [ProductController::class, 'show']);
Route::post('/products', [ProductController::class, 'store']);
```

```
// In app/Http/Controllers/ProductController.php
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    public function show($id)
    {
        $product = Product::find($id);
        return view('products.show', compact('product'));
    }

    public function store(Request $request)
    {
        Product::create($request->all());
        return redirect('/products');
    }
}
```

Benefits:

- ☐ Organized and clean code

- ☐ Easy to maintain and develop
 - ☐ Reusable methods
 - ☐ Easy to test
-

Creating Controllers

1. Creating Controller Manually

You can create a file in `app/Http/Controllers`:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index()
    {
        return 'Product List';
    }
}
```

2. Creating Controller Using Artisan (Recommended)

```
php artisan make:controller ProductController
```

This command creates a file:

```
app/Http/Controllers/ProductController.php
```

3. Creating Resource Controller

```
php artisan make:controller ProductController --resource
```

Creates a Controller with 7 ready methods:

- `index()` - Display list
- `create()` - Show creation form
- `store()` - Save data
- `show($id)` - Display single item
- `edit($id)` - Show edit form
- `update($id)` - Update data
- `destroy($id)` - Delete item

4. Creating Controller with Model

```
php artisan make:controller ProductController --resource --model=Product
```

Creates Controller with automatic Model import.

5. Creating API Controller

```
php artisan make:controller API/ProductController --api
```

Like Resource but without `create()` and `edit()` (APIs don't need them).

Types of Controllers

1. Single Action Controller

A controller with only one action.

```
php artisan make:controller ShowProfileController --invokable
```

```
class ShowProfileController extends Controller
{
    public function __invoke($id)
    {
        $user = User::find($id);
        return view('profile', compact('user'));
    }
}
```

Usage:

```
Route::get('/profile/{id}', ShowProfileController::class);
```

When to use?

- When you have one complex action
- To separate complex logic

2. Resource Controller

A controller for managing full CRUD operations.

```
class ProductController extends Controller
{
    // GET /products
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    // GET /products/create
    public function create()
    {
        return view('products.create');
    }

    // POST /products
    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|max:255',
            'price' => 'required|numeric',
        ]);

        Product::create($validated);
        return redirect()->route('products.index')
            ->with('success', 'Product created successfully');
    }

    // GET /products/{id}
    public function show(Product $product)
    {
        return view('products.show', compact('product'));
    }
}
```

```

}

// GET /products/{id}/edit
public function edit(Product $product)
{
    return view('products.edit', compact('product'));
}

// PUT/PATCH /products/{id}
public function update(Request $request, Product $product)
{
    $validated = $request->validate([
        'name' => 'required|max:255',
        'price' => 'required|numeric',
    ]);

    $product->update($validated);
    return redirect()->route('products.index')
        ->with('success', 'Product updated successfully');
}

// DELETE /products/{id}
public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index')
        ->with('success', 'Product deleted successfully');
}
}

```

Connecting Resource Controller with Routes:

```
Route::resource('products', ProductController::class);
```

This single line creates **7 routes** automatically!

```
php artisan route:list --name=products
```

Method	URI	Name	Action
GET	/products	products.index	index
GET	/products/create	products.create	create

Method	URI	Name	Action
POST	/products	products.store	store
GET	/products/{id}	products.show	show
GET	/products/{id}/edit	products.edit	edit
PUT/PATCH	/products/{id}	products.update	update
DELETE	/products/{id}	products.destroy	destroy

3. API Resource Controller

For APIs, without `create` and `edit`:

```
php artisan make:controller API/ProductController --api
```

```
Route::apiResource('products', ProductController::class);
```

Creates **5 routes** only:

- `index`, `store`, `show`, `update`, `destroy`

4. Nested Resource Controllers

For nested resources (like comments on a post):

```
Route::resource('posts.comments', CommentController::class);
```

Creates routes like:

- `/posts/{post}/comments`
- `/posts/{post}/comments/{comment}`

```
class CommentController extends Controller
{
    public function index(Post $post)
    {
        return view('comments.index', [
            'post' => $post,
            'comments' => $post->comments
        ]);
    }
}
```

```
    });  
  }  
  
  public function store(Request $request, Post $post)  
  {  
    $post->comments()->create($request->all());  
    return redirect()->route('posts.comments.index', $post);  
  }  
}
```

Resource Controllers

Specifying Only Certain Methods

If you don't need all 7 methods:

```
// Only index and show  
Route::resource('products', ProductController::class)  
    ->only(['index', 'show']);  
  
// Everything except destroy  
Route::resource('products', ProductController::class)  
    ->except(['destroy']);
```

Naming Parameters

```
Route::resource('products', ProductController::class)  
    ->parameters([  
        'products' => 'product_id'  
    ]);  
  
// Result: /products/{product_id}
```

Resource Routes with Middleware

```
Route::resource('products', ProductController::class)  
    ->middleware('auth');
```

Dependency Injection

What is Dependency Injection?

Dependency Injection is a technique to automatically inject objects that a Controller needs.

1. Injecting Request

```
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function store(Request $request)
    {
        // Laravel automatically injects Request object
        $name = $request->input('name');
        $price = $request->input('price');

        Product::create([
            'name' => $name,
            'price' => $price
        ]);

        return redirect()->route('products.index');
    }
}
```

2. Injecting Model (Route Model Binding)

```
class ProductController extends Controller
{
    public function show(Product $product)
    {
        // Laravel fetches product from database automatically
        return view('products.show', compact('product'));
    }

    public function update(Request $request, Product $product)
    {
        $product->update($request->all());
        return redirect()->route('products.show', $product);
    }
}
```

```

    }

    public function destroy(Product $product)
    {
        $product->delete();
        return redirect()->route('products.index');
    }
}

```

How does it work?

- Laravel searches for `Product::find($id)` automatically
- If not found, returns 404 automatically

3. Injecting Services

```

use App\Services\PaymentService;

class OrderController extends Controller
{
    public function store(Request $request, PaymentService $payment)
    {
        // Laravel injects PaymentService automatically
        $order = Order::create($request->all());
        $payment->process($order);

        return redirect()->route('orders.show', $order);
    }
}

```

4. Injecting in Constructor

To inject a dependency used in all methods:

```

class ProductController extends Controller
{
    protected $productService;

    public function __construct(ProductService $productService)
    {
        $this->productService = $productService;
    }

    public function index()
    {

```

```

    {
        $products = $this->productService->getAllProducts();
        return view('products.index', compact('products'));
    }

    public function store(Request $request)
    {
        $this->productService->createProduct($request->all());
        return redirect()->route('products.index');
    }
}

```

Benefits of Dependency Injection

□ **Clean and readable code** □ **Easy testing** (can use Mock Objects) □ **Separation of Concerns** □ **Reusability**

Middleware in Controllers

1. Using Middleware in Routes

```

Route::resource('products', ProductController::class)
    ->middleware('auth');

```

2. Using Middleware in Constructor

```

class ProductController extends Controller
{
    public function __construct()
    {
        // All methods require authentication
        $this->middleware('auth');

        // Except index and show
        $this->middleware('auth')->except(['index', 'show']);

        // Only create, store, and edit
        $this->middleware('auth')->only(['create', 'store', 'edit']);
    }
}

```

3. Custom Middleware for Specific Methods

```
class ProductController extends Controller
{
    public function __construct()
    {
        // Check permissions for delete and update only
        $this->middleware('can:update,product')->only(['edit', 'update']);
        $this->middleware('can:delete,product')->only('destroy');
    }
}
```

Best Practices

☐ Do:

1. Use Clear Names

```
// ☐ Good
class ProductController extends Controller
class UserProfileController extends Controller
class AdminDashboardController extends Controller

// ☐ Bad
class MyController extends Controller
class Controller1 extends Controller
```

2. Don't Put Complex Logic in Controller

```
// ☐ Bad
public function store(Request $request)
{
    $product = new Product();
    $product->name = $request->name;
    $product->price = $request->price;
    $product->save();

    // Send email
    Mail::to($user)->send(new ProductCreated($product));
}
```

```

    // Update statistics
    Statistics::increment('products_count');

    // Save to log
    Log::info('Product created: ' . $product->id);

    // 50 more lines...

    return redirect()->route('products.index');
}

// ☐ Good - Use Service Classes
public function store(Request $request, ProductService $service)
{
    $product = $service->createProduct($request->validated());
    return redirect()->route('products.index');
}

```

3. Use Form Requests for Validation

```

// ☐ Bad
public function store(Request $request)
{
    $request->validate([
        'name' => 'required|max:255',
        'price' => 'required|numeric|min:0',
        'description' => 'required',
        // 20 more rules...
    ]);
}

// ☐ Good
public function store(StoreProductRequest $request)
{
    // Validation happens automatically
    Product::create($request->validated());
}

```

Creating Form Request:

```
php artisan make:request StoreProductRequest
```

```

class StoreProductRequest extends FormRequest
{

```

```

public function rules()
{
    return [
        'name' => 'required|max:255',
        'price' => 'required|numeric|min:0',
        'description' => 'required',
    ];
}
}

```

4. Use Route Model Binding

```

// ❌ Bad
public function show($id)
{
    $product = Product::find($id);
    if (!$product) {
        abort(404);
    }
    return view('products.show', compact('product'));
}

// ✅ Good
public function show(Product $product)
{
    return view('products.show', compact('product'));
}

```

5. Use compact() or Array

```

// ✅ Good
return view('products.index', compact('products', 'categories'));

// ✅ Also good
return view('products.index', [
    'products' => $products,
    'categories' => $categories
]);

// ❌ Old (but works)
return view('products.index')->with('products', $products);

```


❌ Don't:

1. Don't Repeat Code (DRY Principle)

```
// ❌ Bad
public function adminIndex()
{
    $products = Product::where('status', 'active')->get();
    return view('admin.products', compact('products'));
}

public function userIndex()
{
    $products = Product::where('status', 'active')->get();
    return view('user.products', compact('products'));
}

// ✅ Good - Use shared method
protected function getActiveProducts()
{
    return Product::where('status', 'active')->get();
}

public function adminIndex()
{
    $products = $this->getActiveProducts();
    return view('admin.products', compact('products'));
}

public function userIndex()
{
    $products = $this->getActiveProducts();
    return view('user.products', compact('products'));
}
```

2. Don't Put Complex Queries in Controller

```
// ❌ Bad
public function index()
{
    $products = DB::table('products')
        ->join('categories', 'products.category_id', '=', 'categories.id')
        ->join('brands', 'products.brand_id', '=', 'brands.id')
        ->select('products.*', 'categories.name as category_name')
        ->where('products.status', 'active')
```

```

        ->orderBy('products.created_at', 'desc')
        ->paginate(10);

    return view('products.index', compact('products'));
}

// ☐ Good - Use Repository or Query Scopes
public function index(ProductRepository $repository)
{
    $products = $repository->getActiveWithRelations();
    return view('products.index', compact('products'));
}

```

Practical Exercises

Exercise 1: Create Simple Controller ☐

```
php artisan make:controller PageController
```

```

class PageController extends Controller
{
    public function home()
    {
        return view('home');
    }

    public function about()
    {
        return view('about');
    }

    public function contact()
    {
        return view('contact');
    }
}

```

Routes:

```

Route::get('/', [PageController::class, 'home'])->name('home');
Route::get('/about', [PageController::class, 'about'])->name('about');

```

```
Route::get('/contact', [PageController::class, 'contact'])->name('contact');
```

Exercise 2: Resource Controller

```
php artisan make:controller ProductController --resource
```

Routes:

```
Route::resource('products', ProductController::class);
```

Implementation in ProductController:

```
class ProductController extends Controller
{
    public function index()
    {
        $products = [
            ['id' => 1, 'name' => 'Laptop', 'price' => 5000],
            ['id' => 2, 'name' => 'Phone', 'price' => 3000],
            ['id' => 3, 'name' => 'Tablet', 'price' => 2000],
        ];

        return view('products.index', compact('products'));
    }

    public function show($id)
    {
        $products = [
            1 => ['id' => 1, 'name' => 'Laptop', 'price' => 5000],
            2 => ['id' => 2, 'name' => 'Phone', 'price' => 3000],
            3 => ['id' => 3, 'name' => 'Tablet', 'price' => 2000],
        ];

        $product = $products[$id] ?? abort(404);

        return view('products.show', compact('product'));
    }

    public function create()
    {
        return view('products.create');
    }

    public function store(Request $request)
```

```

{
    // In real app, save to database
    return redirect()->route('products.index')
        ->with('success', 'Product added successfully');
}
}

```

Exercise 3: Single Action Controller

```
php artisan make:controller ShowDashboardController --invokable
```

```

class ShowDashboardController extends Controller
{
    public function __invoke()
    {
        $stats = [
            'users' => 150,
            'products' => 80,
            'orders' => 320,
            'revenue' => 45000
        ];

        return view('dashboard', compact('stats'));
    }
}

```

Route:

```

Route::get('/dashboard', ShowDashboardController::class)
    ->name('dashboard');

```

Exercise 4: Controller with Dependency Injection

```
php artisan make:controller UserController
```

```

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function store(Request $request)
    {
        // Validate data
    }
}

```

```

        $validated = $request->validate([
            'name' => 'required|max:255',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8',
        ]);

        // In real app: User::create($validated)

        return redirect()->route('users.index')
            ->with('success', 'User created successfully');
    }

    public function update(Request $request, $id)
    {
        $validated = $request->validate([
            'name' => 'required|max:255',
            'email' => 'required|email',
        ]);

        // In real app: $user->update($validated)

        return redirect()->route('users.show', $id)
            ->with('success', 'User updated successfully');
    }
}

```

Exercise 5: Middleware in Controller

```

class AdminController extends Controller
{
    public function __construct()
    {
        // All methods require authentication
        $this->middleware('auth');
    }

    public function dashboard()
    {
        return view('admin.dashboard');
    }

    public function users()
    {
        return view('admin.users');
    }
}

```

```
public function settings()
{
    return view('admin.settings');
}
```

Routes:

```
Route::prefix('admin')->name('admin.')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'dashboard'])
        ->name('dashboard');
    Route::get('/users', [AdminController::class, 'users'])
        ->name('users');
    Route::get('/settings', [AdminController::class, 'settings'])
        ->name('settings');
});
```

Challenge: Complete CRUD System

Create a CRUD system for managing Posts with:

1. Create Controller:

```
php artisan make:controller PostController --resource
```

2. Define Routes:

```
Route::resource('posts', PostController::class);
```

3. Implement Methods:

```
class PostController extends Controller
{
    // Display all posts
    public function index()
    {
        $posts = [
            ['id' => 1, 'title' => 'Introduction to Laravel', 'author' => 'Ahmed'],
            ['id' => 2, 'title' => 'Learning Controllers', 'author' => 'Mohammed'],
        ];
    }
}
```

```

        ];
        return view('posts.index', compact('posts'));
    }

    // Show form to create new post
    public function create()
    {
        return view('posts.create');
    }

    // Save new post
    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|max:255',
            'content' => 'required',
        ]);

        // Post::create($validated);

        return redirect()->route('posts.index')
            ->with('success', 'Post created successfully');
    }

    // Display single post
    public function show($id)
    {
        $post = [
            'id' => $id,
            'title' => 'Introduction to Laravel',
            'content' => 'Post content...',
            'author' => 'Ahmed'
        ];
        return view('posts.show', compact('post'));
    }

    // Show edit form
    public function edit($id)
    {
        $post = [
            'id' => $id,
            'title' => 'Introduction to Laravel',
            'content' => 'Post content...'
        ];
        return view('posts.edit', compact('post'));
    }

    // Update post

```

```
public function update(Request $request, $id)
{
    $validated = $request->validate([
        'title' => 'required|max:255',
        'content' => 'required',
    ]);

    // $post->update($validated);

    return redirect()->route('posts.show', $id)
        ->with('success', 'Post updated successfully');
}

// Delete post
public function destroy($id)
{
    // $post->delete();

    return redirect()->route('posts.index')
        ->with('success', 'Post deleted successfully');
}
}
```

□ Summary

In this lesson, you learned:

□ MVC pattern and its importance □ What controllers are and why we use them □ Creating Controllers in different ways □ Types of Controllers (Single Action, Resource, API) □ Resource Controllers and the 7 methods □ Dependency Injection □ Using Middleware in Controllers □ Best practices

□ Additional Resources

- [Laravel Controllers Documentation](#)
- [Resource Controllers](#)
- [Dependency Injection](#)

□ Test Yourself

Before moving to the next lesson, make sure you can answer:

1. What's the difference between Model, View, and Controller?
2. Why use Controllers instead of putting code in routes?
3. What's the difference between Resource Controller and API Resource Controller?
4. What is Dependency Injection?
5. How do you use Middleware in Constructor?

Next Lesson

Ready for more? Move on to [Lesson 4: Blade Templates and User Interfaces](#)

In Lesson 4, you'll learn:

- Blade templating engine
- Directives
- Components
- Layouts
- And more!

Happy Learning! □